

Nonlinear gadgetization

The product-share encoding, described from scratch

2026-07-25

Written for a reader who knows the linear (XOR-pair) gadgetizer and nothing else about this line of work. Covers the design of 2026-07-23 and the revisions of 2026-07-24 as one piece. Option reference in §9.

1 Where linear gadgetization leaks

The gadgetizer takes a small circuit C and produces a much larger equivalent G that computes the same function on its low wires. Every logical value of C is **secret-shared** across the wires of G , and G 's gates manipulate the shares. The security goal is that G reveal nothing about C beyond its input/output behaviour—in particular that an adversary looking at a point of G cannot tell **where in C 's computation** that point sits.

In the linear gadgetizer a logical value v lives on two carrier wires as $v = c_0 \oplus c_1$. This is the whole problem. The decode is linear, so **at every instant, every logical value of C is a degree-1 (affine) function of G 's wires**. An adversary who fits affine functions can therefore read C 's intermediate state out of G at any point—no matter how algebraically complex that state is as a function of the input.

`hmap_affine` measures this directly. For each prefix i of C and prefix j of G , it fits the best GF(2) affine reconstruction of C 's state after i gates from G 's wire values after j gates, and reports held-out error H (0 = perfectly recoverable, 0.5 = hidden). Under linear sharing the aligned cells light up and a low- H **diagonal** runs across the map. That diagonal is a picture of C 's computational progress inside G .

Read these maps by the **ridge**, never by the mean (which saturates near 0.5): *depth* is the per-row prominence of the low- H valley above its row background, ρ is the Spearman correlation of row index against ridge location (“is it a monotone diagonal”), and *perm-z* scores that against a shuffled null.

The diagonal survived everything. Local mixing (ssg, fmix), generation targeting to generation 100+, four million mixing moves, growth phases: all of them shallow the valley somewhat but leave $\rho = 1$. The reason is structural—mixing re-randomizes *which* wires carry the sharing, but the value stays affine in the new wires, so an affine adversary just re-fits. Masking values *between* their uses does not help either: the ridge lives **at** the use points, and any scheme that un-masks an operand so an ordinary gate can read it re-exposes exactly there. (An earlier “deferred mask” design failed on precisely this.)

The leak is not a mixing deficiency. It is the linear decode.

2 The fix: a balanced nonlinear decode

Give each logical value, on top of its XOR pair, a set of permanent **multiplicative mask terms**:

$$v_i = w_u \oplus w_{u'} \oplus \bigoplus_j \prod_l (w_{p_{jl}} \oplus a_{jl}) \oplus c_i$$

Each $\prod_l (w \oplus a)$ is a conjunction of literals over a dedicated band of wires; c_i is a bookkeeping constant held in a ledger, never on a wire. Now the decode is nonlinear, and an affine adversary cannot express it.

Why the XOR pair is still there. A pure product decode is un-gateable, and the reason is a counting obstruction worth stating: $(w_p \oplus a)(w_q \oplus b)$ partitions its representations into classes of size 3 and 1, and no reversible gadget can conditionally flip between classes of *unequal* size—a bijection cannot map a 3-element class onto a 1-element class. This holds for every fixed (a, b) , with any ancillae, garbage or re-encoding. The linear part re-symmetrizes the classes; it is structural, not a convenience.

3 The frozen source band

The mask sources live on a band of extra wires appended after the $2n$ carriers. Each band wire is filled at the input port and its **value** is then never changed again. Because the sources are frozen, every registered mask term is **time-invariant**—unaffected by all re-randomization and all gate traffic—so a mask is one gate, not a maintained data structure, and nothing is ever flushed except the final unshare.

The fill must not be linear. The obvious fill, a random XOR of data wires, makes every band wire an *affine invariant of the entire circuit*, held from the input port to the output port. That is exactly the class of relation which **collapses the preimage SAT search**: an attacker who learns such invariants from forward runs and injects them turned $n=32$ instances from “>2 hours, timeout” into “~10 minutes”. A linear band fill manufactures them for free.

The fill is therefore

$$band_j = junk \oplus x_{\text{pivot}} \oplus (\text{small linear part}) \oplus \bigoplus^M (\text{two-source products})$$

with each product’s two sources drawn from the data wires **and from already-filled band wires**. The cascade is the point: multiplying two already-balanced band bits keeps the firing rate near 1/4 while the input degree multiplies up the band, so high degree in x is reached with nothing wider than a two-control gate. (A flat high-degree monomial would fire on only a 2^{-d} fraction of inputs and behave statistically like its linear part.)

Balance is exact, which the mask argument needs: the fresh pivot is excluded from the rest of that wire’s transitive support, so the fill is balanced over uniform x for any nonlinear part.

The same fill is emitted again after the output bookend (**mirror** F'), now over every non-output wire rather than the band’s own, so the fill’s target set does not itself publish where the band is.

The band rolls. A frozen value on a fixed wire is a wire that is read constantly and written twice, which a one-pass census of the emitted gate list separates from the carriers perfectly—writes per wire 10–22 against the carriers’ 209–1004, reads 9,725–14,712 against 272–804. Identifying a mask source is the first thing a restriction adversary must do, and that handed over the list. So

the *role* moves: one **roll** per inter-SG gap swaps a band variable’s wire with a uniformly chosen other wire—another band wire, or a carrier, in which case the two exchange roles and the vacated wire becomes a live carrier. The bookkeeping re-points (`GadgetState::pairs` for the carrier, the ledger’s location map for the band), so every decode is invariant; only the address changes. Mask slots therefore name band *variables*, not wires, and resolve to literals at emission time.

After rolling, no threshold on writes, on reads, or on gate shape separates band from carriers, and no wire is unwritten across any 50k-gate window. The swap is emitted in a mixed vocabulary—either a CNOT or the pair $t \oplus = s \wedge u$, $t \oplus = s \wedge \neg u$, which sums to s exactly—because with plain CNOTs the write *counts* matched but the gate *shapes* did not: nothing but a roll writes a wire while it holds a band variable. Residual: a 10k-gate window still leaves about a dozen wires unwritten, so an adversary who stitches windows together and follows the swaps can still track the band. Cost $\sim 12\%$ of the gadget’s gates.

4 The share-native fold—the part that actually matters

An encoding is only as good as the gate that operates on it. If a gate reconstructs its operands to compute with them, the ridge comes straight back at the use point.

So the gadget never reconstructs. A source gate $v_t \oplus = f(\text{controls})$ is applied by **expanding f ’s ANF over the operands’ full decodes**. Each control value contributes a set of atoms—its two carrier literals, its mask terms, and a constant when the ledger constant or a negative control polarity applies—and every cross-product of one atom per control becomes **one conjunction fragment** written into a randomly chosen carrier of the target.

The property that matters: a value’s own mask term appears only *inside* a fragment’s firing condition, as control literals. **It is never materialized on a wire**. No fragment computes a value’s two-carrier XOR, and no wire is written except the target’s carriers. Operands stay masked straight through the gate. Constants and the gate’s complement go to the ledger constant, so the encoding emits no bare X anywhere.

Fragments are emitted in randomized order. Without this, each source gate produced its $(k + 2)^2$ fragments consecutively, in deterministic order, all aimed at one value’s two carriers—a **progress clock readable statically**, with no execution required.

5 Degree and count do different jobs

Two independent levers, and conflating them wastes gates.

Degree hides algebraically. A degree- d mask term is not in the degree- $< d$ $\text{GF}(2)$ span, so a *single* one forces $H = 0.5$ against **every** reconstruction adversary of degree $< d$, regardless of how many terms there are.

Count hides statistically. Against a best-approximation adversary, k stacked terms pile up: the best affine readout error tends to $1/2$ as $1/2 - 2^{-(k+1)}$.

This yields **towers**: a degree- d mask kills every adversary below degree d , and the diagonal returns at degree exactly d . The wall moves up one level per $+1$ of mask degree; it does not vanish. Verified with real degree-2 and degree-3 adversaries, each capability-controlled.

A degree- d term is also a **sparser** perturbation (it flips the value only 2^{-d} of the time), so wider masks are individually *weaker* statistically even as they are stronger algebraically. That argued for a mixed design—low-degree terms for statistical margin plus tower terms for algebraic hiding—and the plan was [2, 3, 3].

The production plan is now [3, 3]: two degree-3 terms, no degree-2 base term. Three measurements decided it.

Two terms is the floor. [3]—a single degree-3 term—puts the degree-1 diagonal straight back ($\rho = 0.995$, median row prominence 0.027), even though the tower argument says one degree- d term should hide from every adversary below d . Whatever the mechanism, one term does not do it.

The base term costs a third of the fold. Fragments per source gate go as $(2 + k_{\text{total}})^{\text{arity}}$ and 91% of the source is arity 2, so dropping the third term removes 33% of the fragments and 25% of the gadget: 247,326 gates against 318,813 at $n=128$.

And it buys little against a degree-2 adversary. Against a strictly linear predictor the base term is worth 0.09 (best-predictor agreement 0.609 for [2, 3, 3] versus 0.703 for [3, 3], floor 0.525). But the term *is* a product of two band wires, so one AND cancels it: allowing the predictor a single $(w_p \oplus a)(w_q \oplus b)$ takes [2, 3, 3] from 0.602 to 0.693 while [3, 3] only moves $0.718 \rightarrow 0.728$. The gap between the plans falls to 0.036. (With the band rolling, that AND search must first find the band, which buys the rolling build a further margin.)

Raising the tower is nearly free in gate count (fragment *count* is set by the term count and the source gate’s arity, not by mask degree—widening a mask only adds literals to existing fragments). What it costs is fragment **width**.

6 Gate width, and a corrected assumption

Folding a degree- d mask through a w -operand gate yields fragments of width up to $d \cdot w$, so the production gadget contains gates of width up to 6. Two consequences were expected to be bad, and exactly one of them is.

Real: with `--db-ctrl-cap 2`, any gate with ≥ 3 controls is ineligible for frozen-DB re-encoding. About 62% of the gadget is in that class. This is why generation targeting cannot report progress here (§8)—and, in the other direction, why the encoding **resists DB-based simplification**, which is a property one wants of an obfuscation.

Not real: the assumption that wide gates are immobile fossils. A per-width commutation-leeway census of the real $n=128$ gadget says the opposite—the width-5 and width-6 gates are the **most mobile gates in the circuit** (mean float-box 2779 and 2778, versus 694 for the g57s). A gate with many literals has more chances to hold an *opposite* literal against a neighbour, which exempts the pair from colliding, so it crosses more freely.

A mode exists that removes wide gates entirely (`--prod-max-width`, §9): it ladders every emission down to two controls using a Barenco double sweep over **dirty borrowed carriers**—arbitrary-valued wires, each visited an even number of times so its dirty content cancels between readings. It costs no extra wires and preserves correctness under arbitrary junk. But it costs **6.4×** the gates and *reduces* mobility to mean 259, so it is off by default. Its one live justification is DB re-encodability, and whether that is worth 6.4× is not yet measured.

7 The slice block

The gadget computes C on its low n wires for *arbitrary* junk on every other wire. A block prepended at the input port is what turns that into “computes C only when the non-data wires are zero”, and it is also what stops the inverse circuit from handing out C^{-1} .

The band is part of the slice. It was not, and that was a one-query distinguisher: with the band outside the block, setting a band wire to 1 provably changed nothing, so comparing $G(x, s)$ with $G(x, 0)$ sorted the non-data wires into “matters” (aux) and “does not” (band) in a handful of queries—and the band is where every mask source lives. The slice is now **every** non-data wire, aux and band alike.

The construction. All gates are positive-polarity, target a data wire, and read exactly one slice wire, in three shapes:

$$x_t \oplus = s_w, \quad x_t \oplus = x_a \wedge s_w, \quad x_t \oplus = x_a \wedge x_b \wedge s_w,$$

in a 1:1:1 mix, with the target and the data controls drawn freely over all n data wires and the whole list uniformly shuffled. On the all-zero slice the slice control kills every gate individually, so the block is the identity there gate by gate—no ordering or pairing argument needed.

The three-control shape is load-bearing. With at most one data control per gate, fixing any slice turns every gate into a constant flip or a transvection, and those compose to an **affine** map: $S(\cdot, s)$ would have degree 1 in x and be pinned by $n+1$ queries, so the “looks like a random function of both arguments” intent would hold in the s direction only.

Only one regularity is imposed: slice wires are covered by a balanced round-robin. A slice wire that no gate reads is a provably fixed slice, and free draws leave one uncovered a few percent of the time at these budgets.

The guarantee is checked, not proved. “No nonzero slice is fixed” is a property of the draw, so the block is verified and redrawn until it holds: exhaustively over every slice and every input where the space fits ($n + \text{band} \leq 20$, which is exactly the regime where wrong-slice fixes were ever observed), and otherwise by sampling every single-wire slice, many weight-2 slices and random slices against 64 bit-sliced random inputs each. The sampled check is a spot check, not a proof; at production width the honest statement is the measured one—a slice is fixed only if its fired subsequence composes to the identity, seen in $\approx 4 \cdot 10^{-3}$ of draws at $n=3$ and $0/50k$ by $n=8$, decaying fast in n .

7.1 An exactly-pinning two-layer variant, and why it is not used

The property *can* be made a theorem. Split the data wires into disjoint pools—targets T , data controls R —with every gate targeting T and reading only R . Then on a fixed slice each fired gate adds $e_t m(x_R)$ for a monomial $m \in \{1, x_c, x_a x_b\}$; no gate can change another’s monomial, so they all commute and

$$S(x, s) = x + \sum_{(t,m)} q_{(t,m)}(s) e_t m(x_R),$$

with every q a *linear* form in s . “Identity on slice s ” is then a linear system, certified by a rank check on the emitted gates and seeded by one pin gate per slice wire—order-free, so the uniform shuffle stays free.

One layer has two defects: R is never disturbed, and the whole disturbance is a function of x_R *alone* (nine bits of the input at production sizing), so an adversary who sets $x_R = 0$ collapses the block

to a translation. **Two layers repair both.** Run pools (T_1, R_1) entirely, then $(T_2, R_2) = (R_1, T_1)$. Layer 2 never writes T_1 , so

$$z_{T_1} = x_{T_1} + F_1(x_{R_1}), \quad z_{R_1} = x_{R_1} + F_2(x_{T_1} + F_1(x_{R_1})),$$

and $z = x$ forces $F_1 \equiv 0$ from the first line, then $F_2 \equiv 0$ from the second. The conditions decouple, the certificate is the union of both layers’ rows, the pins split across layers, and the gate budget is **unchanged**. Every data bit then influences the disturbance, and the input set that switches the nonlinearity off shrinks from a $2^{n-|R|}$ subcube to a single point.

It is not used, because of the mixing barrier. Layer-2 gates read precisely what layer-1 gates write, so the two halves do not commute and cannot be shuffled into each other. That ordering survives the final commuting shuffle and every downstream mixing move that preserves the relative order of colliding pairs: the block would carry a permanent two-phase seam that no amount of mixing can dissolve. It would be bought to exclude a single failure—a nonzero slice that also computes C —whose payoff to an adversary is unclear, since C is computable on the honest slice anyway. Structure that mixing cannot erase is the more expensive side of that trade. The unstructured block stands; its guarantee is checked rather than proved.

8 What it achieves

All heatmaps read by the ridge measure with verified references, on a real $n=128$ sliced sandwich (source $C = 3000$ g57 gates).

gadget	adversary	depth	depthMed	ρ	meanH
linear (plain), 85,354 g / 512 w	degree 1	0.356	0.363	1.000	0.475
[3, 3] + roll, 247,326 g / 558 w	degree 1	0.043	0.000	0.015	0.498
[2, 3, 3], 281,436 g / 568 w	degree 1	0.043	0.000	0.018	0.498
[2, 3, 3], 281,436 g / 568 w	degree 2	0.043	0.000	−0.007	0.498
[3, 3], band-restricted	degree 2	0.044	0.004	0.031	0.498

A second instrument now sits beside the ridge. The exact-span measure asks whether C_i ’s bit is an *exact* function of G_j ’s wires and scores 0.5 whenever it is not, so it is blind to bias—and bias is all the term count controls. `hmap_stat` measures the best *approximate* predictor instead (every single wire, every XOR of two wires, optionally one AND term), against a null floor from the same search on a random target. Median per-row best agreement at $n=128$: plain gadget 0.907, [2, 3, 3] 0.609, [3, 3] 0.703, [3] 0.803, floor 0.525—tracking the piling-up prediction a few points low, as a two-wire family on a coarse grid must. Every such number is a lower bound.

The median original-circuit prefix has **zero** ridge prominence and the reconstruction is uncorrelated with progress. The nulls are capability-controlled: a design whose masks are only degree 2 *is* recovered by the same degree-2 adversary at $\rho = 1.00$, so these are genuine hiding, not a weak measurement.

Earlier work carried the whole pipeline through—gadget $\rightarrow$ a DB-mixing phase $\rightarrow$ threefold growth—and the diagonal stayed dead **identically** at every stage. Mixing neither helps nor hurts, because the leak is gone before mixing begins.

Cost. The encoding multiplies the plain gadget by $\sim 2.9\times$ in gates and adds a band of extra wires (46 at $n=128$, versus 512 carriers). The fold is **76% of the emitted gadget**, and per source gate

it emits about $(k_{\text{total}} + 2)^{\text{arity}}$ fragments. With 91% of the source at arity 2 the term count is a quadratic lever and everything else is second order; the rolling band is the only other item above a few percent, at $\sim 12\%$.

9 Reading a mixing run on this material

Generations only advance under DB re-encoding, and $\sim 62\%$ of a product-share gadget is width ≥ 3 , hence cap-ineligible: those gates sit at generation 0 forever. **G=** is therefore measured over the **targetable** population—cap-eligible and not written off—which is exactly the set the DB channel can move. **Gall=** reports the older all-gates percentile beside it, and on this material that one is **structurally pinned at 0**: it is not a progress number here, and no amount of mixing moves it.

So “run to generation N ” means N over the targetable material. **--gen-target N** drives it, **--gen-stop-frac** stops on lag/tgtbl, and **--gen-giveup** retires eligible gates the DB genuinely cannot reach. Read **G=**, **lag=** and **tgtbl=**; **wlag=** is the wide population, **Gall=** the legacy figure.

(Before the fix in 512ce31c both **G=** and the dose stop counted all gates, so on this material the stop could never fire and phase A would burn its whole move budget after the dose was complete—bound it with **--moves** if running an older build.)

Also required: **--no-local-verify**. The exhaustive per-rewrite check caps support at 24 wires and **panics** on two wide gates. The whole-circuit **--verify-every** check still runs.

10 Options

On **sss --cnot --gadgetize**; **gen_sandwich_gadget** reads the same settings from the upper-case environment variables. With both counts at 0 the encoding is off and the gadget is bit-identical to the linear build.

flag	env var	default	meaning
--prod-k	PROD_K	0	base mask terms per value (statistical)
--prod-deg	PROD_DEG	2	literals per base term, min 2 (algebraic)
--prod-k-hi	PROD_K_HI	0	tower terms per value
--prod-deg-hi	PROD_DEG_HI	3	tower degree; two deg-3 kill the deg-2 diagonal
--prod-band	PROD_BAND	0 (auto)	source-band width in extra wires
--prod-rsrc	PROD_RSRC	1	mask re-source moves per inter-SG gap
--prod-max-width	PROD_MAX_WIDTH	0 (off)	ladder to this width; 2 = all DB-eligible. $\sim 6.4\times$ gates
--prod-fill-nl	PROD_FILL_NL	0 (linear)	nonlinear cascaded band fill. Effectively free
--prod-roll	PROD_ROLL	0 (off)	band relocations per inter-SG gap; 1 recommended. $\sim 12\%$ g

Mutually exclusive with the deferred-mask **--mask-cov** (RG4). Auto band width is $\approx \max(\sqrt{4nk_{\text{total}}}, 6, \text{deg}_{\text{max}} + 3)$

Production ([3, 3], band auto = 46 at $n=128$):

```
sss --cnot --gadgetize --slice-zero-ccnot \
    --prod-k 0 --prod-k-hi 2 --prod-deg-hi 3 --prod-fill-nl 2 --prod-roll 1
```

Band width is the cost of the degree-3 adversary the tower deliberately admits: $\binom{b}{3}$ product regressors, so 15,180 at $b=46$ against 220 at the slot-space minimum $b=12$ —and at $b=12$ that adver-

sary reads a perfect $\rho=1$ diagonal back off the gadget. Do not shrink the band to its minimum; `--prod-band` can raise it for about ten fill gates per wire.

11 Known gaps

- **The statistical readout only searches two wires.** It exists now (`hmap_stat`, §7), which is what priced $[2, 3, 3] \rightarrow [3, 3]$, but its predictor family is one or two wires plus optionally one AND term. A greedy or LMS fit over more terms would tighten every number, all of which are lower bounds.
- **Rolling leaves a windowed residual.** Over any 50k-gate window no wire is unwritten, but a 10k-gate window still leaves about a dozen—the variables that did not roll—so an adversary stitching windows and following swaps can still track the band. Roll-rate tuning against a windowed adversary, and a schedule that rolls a variable just before its next read, are open.
- **Static distinguishers are instrumented only ad hoc.** The write / read / gate-shape census that caught the body-static band was written for that pass, not by a standing tool, and the fold-order leak was found by reading code. Everything else we measure is execution-based.
- **The degree-4 tower** needs scratch decomposition of width-8 fragments.
- **The DB match-rate cost** of width-6 gates is predicted by the (m, k, c) grid law but never measured directly.